Distributed Training of Neural Networks using MPI

Allen James Vinoy
Cornell University
av642@cornell.edu

Aryan Patel
Cornell University
ap2568@cornell.edu

Abstract

Deep Learning & Neural Networks are arguably the foundations of modern-day AI. Neural Networks often need to be deep with a large number of parameters to exhibit desired results. Distributing the training of Neural Networks across nodes enables the desired scale. However, there is a communication cost associated with distributed training, and several popular architectures, such as Data Parallelism, Model Parallelism, Parameter Server Model, etc., have been designed to address it. This project aims to explore the distributed training of small-scale neural networks and evaluate its performance across four variables, viz., number of layers, model parameters, data batch size, & number of nodes.

Keywords

Neural Networks, Distributed Training, MPI

1 Hypothesis

The primary question we seek to answer is: for a given neural network and target accuracy, in what regimes of model size, batch size, and process count does model-parallel SGD provide useful speedup, and when does communication dominate? We hypothesize that for a fixed model, there exists a range of process counts and batch sizes where strong scaling is close to linear, beyond which MPI communication overhead leads to diminishing returns. We further expect that larger models (more layers and parameters) will exhibit better scaling behavior than smaller ones, due to a higher compute-to-communication ratio.

2 Context

The project work is limited to this class. Additionally, we are referring to material learned in CS 6787 regarding distributed ML techniques. For the scope of this project, we are interested in evaluating performance metrics on distributed training only and not inference.

3 Key Prior Work

Zhao et al. [1] discuss the Fully Sharded Data Parallel model(FSDP), which forms the primary inspiration in terms of relevance of this project. PyTorch FSDP is a robust and generalized solution that leverages many underlying PyTorch mechanisms, such as Tensors and CUDA memory optimizations. The approach of Downpour SGD [2] discusses an asynchronous method of updating gradients and pioneered the distributed parameter server model. An interesting observation from the training speed-up experiment is that for the speech model with 42M parameters, a saturation was observed beyond 8 machines. This implies the optimum count of machines is closely tied with model size. Furthermore, Li et al. [3] formalized the parameter server distributed model mentioned in [2] across various computational problems.

4 Empirical Methodology

Through this project, we aim to understand the concept and experiment with the distributed training of neural networks using C++, BLAS Matrix Multiplication, and MPI without PyTorch support. The Neural Network Architecture will be a Multi-Layer Perceptron with a configurable number of hidden layers, approximately ranging from 2 to 10, with around 100 neurons per layer. The evaluations will be performed on the MNIST dataset. As a benchmark, we will be evaluating the serial time taken across a certain set of model configurations. For experiment 1, we would evaluate the parallel speedup for a fixed model configuration sharded proportionally across process configuration of 1,2,4,8 etc. For experiment 2, we would evaluate parallel speedup various data batch sizes across various process configurations. For experiment 3, we would evaluate parallel speedup across various model configurations tweaking number of layers and model parameters given an empirically appropriate amount of processes based on previous experiments.

The above experiments are currently designed to be run on multiple cores ideally across multiple nodes on Perlmutter by which MPI communication to cost ratio can be effectively evaluated.

5 Challenges and Obstacles

Given the model size we have chosen, there is a risk of distributed computing not proving worthwile at such a small scale. In that case we might have to try increasing the scale of parameters and execute the model on GPUs.

References

[1] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. *Proceedings of the VLDB Endowment*, 16, 12 (2023), 3848–3860.

[2] Jeffrey Dean, Greg S. Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Quoc V. Le, Mark Z. Mao, Marc'Aurelio Ranzato, Andrew Senior, Paul Tucker, Ke Yang, and Andrew Y. Ng. 2012. Large Scale Distributed Deep Networks. In *Advances in Neural Information Processing Systems 25 (NeurIPS '12)*. 1223–1231.

[3] Mu Li, Li Zhou, Zichao Yang, Aaron Li, Fei Xia, David G. Andersen, and Alexander Smola. 2014. Parameter Server for Distributed Machine Learning. *Carnegie Mellon University Technical Report.*